



Univerzitet u Novom Sadu
Fakultet tehničkih nauka



Dokumentacija za projektni zadatak

Studenti: Jorgaćević Lazar SV16/2024
Koprivica Aleksandar SV24/2024
Milojević Stefan SV9/2024
Popović Ognjen SV64/2024

Predmet: Nelinearno programiranje i evolutivni algoritmi

Broj projektnog zadatka: 4.

Tema projektnog zadatka: Genetski algoritam, job shop problem (JSP)

1. Opis problema

Job shop problem (JSP) je problem optimizacije koji se javlja u proizvodnim procesima, gde se određeni broj poslova (jobova) mora obraditi na određenom broju mašina. Cilj je pronaći optimalan raspored obrade poslova na mašinama kako bi se minimizirao ukupno vreme završetka (makespan) ili neki drugi kriterijum performansi. Svaki posao se sastoji od niza operacija koje moraju biti obrađene na određenim mašinama, a svaka operacija ima svoje trajanje.

Problem je poznat po svojoj složenosti i NP-težini, što znači da nije poznat efikasan algoritam koji može rešiti sve instance problema u polinomijalnom vremenu. Zbog toga, često se koriste heuristički i metaheuristički algoritmi, poput genetskih algoritama, kako bi se pronašla dobra rešenja u razumnom vremenskom okviru.

Precizna formulacija problema koji je ovde rešen je sledeća: Neka je $J = \{J_i : i \in \{1, \dots, n\}\}$ skup poslova, a $M = \{M_j : j \in \{1, \dots, m\}\}$ skup mašina. Svaki posao J_i se sastoji od niza od n_i operacija $O_i = (O_{i1}, O_{i2}, \dots, O_{in_i})$, a svaka operacija je određena parom (M_j, t_{ij}) , gde M_j predstavlja mašinu na kojoj se operacija obrađuje, a t_{ij} je vreme trajanja operacije. Za $O = \{O_i : i \in \{1, \dots, n\}\}$ skup svih operacija, neka je funkcija $s : O \rightarrow \mathbb{N}_0$ funkcija koja dodeljuje početno vreme svakoj operaciji, tako da su zadovoljeni sledeći uslovi:

1. Sve mašine su dostupne od početka, tj. $s(O_{i1}) \geq 0 \quad \forall i \in \{1, \dots, n\}$.
2. U određenom trenutku, svaka mašina može obrađivati najviše jednu operaciju, tj.

$$M(O_{ij}) = M(O_{i'j'}) \Rightarrow s(O_{ij}) + t_{ij} \leq s(O_{i'j'}) \vee s(O_{i'j'}) + t_{i'j'} \leq s(O_{ij})$$

3. Operacije jednog posla se moraju redom izvršavati, tj.

$$s(O_{ik}) + t_{ik} \leq s(O_{i(k+1)}) \quad \forall i \in \{1, \dots, n\}, k \in \{1, \dots, n_i - 1\}$$

Cilj je naći $s_{\min}$ koji minimizira kriterijum optimalnosti (u ovom slučaju, ukupno vreme završetka, tj. makespan) definisan kao

$$C_{\max} = \max\{s(O_{in_i}) + t_{in_i} : i \in \{1, \dots, n\}\}$$

2. Rešenje problema

2.1. Struktura programa

```
usage: job_shop.py [-h] [--jobs_file JOBS_FILE] [--population_size POPULATION_SIZE]
                  [--generations GENERATIONS] [--mutation_count MUTATION_COUNT]
                  [--tournament_size TOURNAMENT_SIZE] [--keep_perc KEEP_PERC]
```

Genetic Algorithm for Job Shop Scheduling

options:

```
-h, --help                show this help message and exit
--jobs_file JOBS_FILE      Path to the jobs file (default: jobs.txt)
--population_size POPULATION_SIZE
                           Population size (default: 150)
--generations GENERATIONS
                           Number of generations (default: 2000)
--mutation_count MUTATION_COUNT
                           Mutation count (default: 20)
```

```
--tournament_size TOURNAMENT_SIZE
                        Tournament size for selection (default: 5)
--keep_perc KEEP_PERC
                        Percentage of best individuals to keep (elitism) (default:
10)
```

Program `job_shop.py` rešava problem job shop scheduling koristeći genetski algoritam. Ulaz programa je tekstualni fajl formata:

```
n m
t_11 M_11 t_12 M_12 ... t_1n_1 M_1n
t_21 M_21 t_22 M_22 ... t_2n_2 M_2n
...
t_n1 M_n1 t_n2 M_n2 ... t_nn_n M_nn
```

gde su n i m redom broj poslova i mašina, a t_{ij} i M_{ij} su vreme trajanja i mašina na kojoj se obrađuje j -ta operacija i -tog posla. Program zatim koristi genetski algoritam da pronade optimalan raspored obrade poslova na mašinama, minimizirajući ukupno vreme završetka (makespan). Rezultat programa je optimalan raspored i pripadajući makespan. Pored toga, program generiše vizualizaciju rasporeda u obliku *Ganttovog dijagrama*, što omogućava lakše razumevanje i analizu rešenja, ali i graf konvergencije genetskog algoritma, koji prikazuje kako se rešenje poboljšava tokom iteracija algoritma.

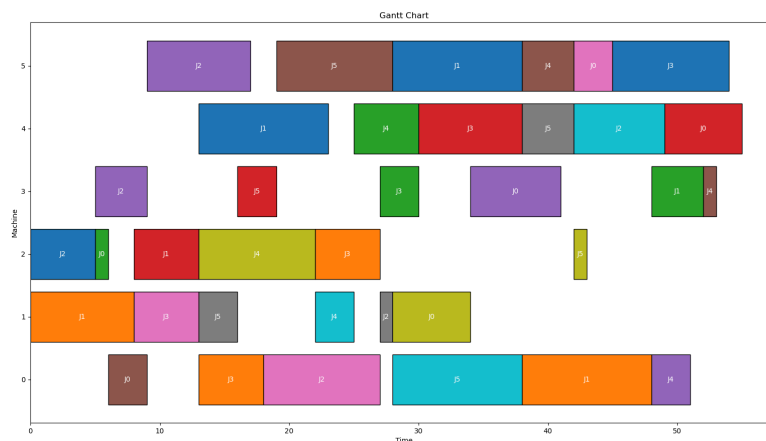


Figure 1: Ganttov dijagram rasporeda poslova na mašinama za instancu ft06.

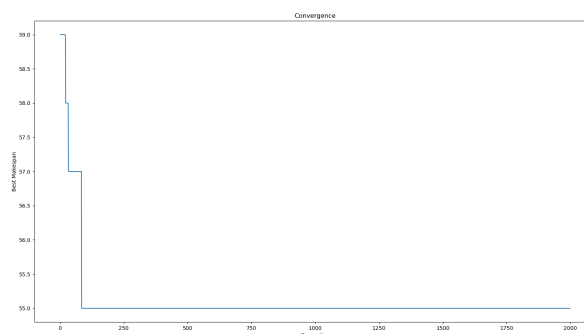


Figure 2: Graf konvergencije genetskog algoritma za instancu ft06.

2.2. Genetski algoritam

- *Hromozom* je predstavljen kao niz indeksa poslova, gde se svaki posao pojavljuje onoliko puta koliko ima operacija. Funkcija `decode()` je zadužena za dekodiranje hromozoma u konkretan raspored operacija određenih torkom (`job_id`, `machine`, `start`, `finish`). Ona se poziva prilikom evaluacije pojedinca i dekodiranja u svrhe prikaza rezultata.
- Za *kriterijum optimalnosti* uzeto je ukupno vreme završetka (*makespan*), koje se računa kao maksimalno vreme završetka svih operacija.
- *Selekcija* se vrši *turnirskom selekcijom*, gde se nasumično bira nekoliko pojedinaca iz populacije, a najbolji među njima se bira za reprodukciju. Korisnik može da podešava veličinu turnira putem parametra `--tournament_size`. Defaultna vrednost je 5.
- *Mutacija* se vrši tako što se nasumično odabere nekoliko pozicija u hromozomu i zamene se sa drugim nasumično odabranim pozicijama. Broj mutacija po pojedincu se podešava putem parametra `--mutation_count`, sa defaultnom vrednošću od 20.
- *Ukrštanje* se vrši *Order Crossover-om* (OX), koji je pogodan za probleme permutacije poput job shop scheduling. Funkcioniše tako što se nasumično odabere segment hromozoma od jednog roditelja, a zatim se preostali poslovi popunjavaju redosledom iz drugog roditelja, preskačući već uključene poslove.
- *Elitizam* je implementiran tako da se određeni procenat najboljih pojedinaca (definisan parametrom `--keep_perc`, sa defaultnom vrednošću od 10%) direktno prenosi u sledeću generaciju, čime se osigurava da se najbolja rešenja ne gube tokom evolucije.

3. Zaključak

Testiranje je obavljeno na par standardnih instanci problema job shop scheduling (Fisher i Thompsonove, Lawrenceove instance), koje su široko korišćene u literaturi za evaluaciju algoritama za ovaj problem. Za manje instance (npr. ft06), genetski algoritam je uspešno pronašao optimalna rešenja, dok su za veće instance (npr. la40) postignuta rešenja sa relativnom greškom u odnosu na optimalna rešenja $< 13\%$. Ovi rezultati su u skladu sa očekivanjima, s obzirom da je trenutna implementacija primitivna. Na osnovu ovih testiranja, eksperimentalno su odabarni parametri algoritma.

Instanca	Najbolji	Najgori	Medijana	Srednja vrednost	BKS	RPD
ft06	55	58	55	55.37	55	0.67%
ft10	976	1097	1016	1013.93	930	9.03%
ft20	1190	1322	1241	1243.57	1165	6.74%
la02	655	695	667	668.23	655	2.02%
la04	590	620	607	602.60	590	2.14%
la16	979	1035	995	995.07	945	5.30%
la22	981	1080	1032	1028.60	927	10.96%
la24	990	1087	1044	1041.97	935	11.44%
la26	1309	1391	1352	1351.97	1218	11.00%
la30	1439	1589	1493	1502.17	1355	10.86%
la32	1916	2105	1995	1990.80	1850	7.61%
la36	1369	1522	1428	1439.77	1268	13.55%
la40	1338	1489	1387	1390.87	1222	13.82%